# Implementation Plan, PMP, WFA Classification, and the RDMA-Compute Interface

Adaptive RDMA Optimization for Distributed LLM Training and Inference on Consumer Hardware

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

March 31, 2026

## Contents

**1 Problem Context** **3**
1.1 The KV Cache Problem . . . 3
1.2 What This Update Adds . . . 3

**2 Theoretical Foundation** **3**
2.1 The Scaling Book's Decode Latency Model . . . 3
2.2 PMP: Bang-Bang QP Allocation . . . 4
2.3 WFA vs. PMP: Complementary Roles . . . 4

**3 Implementation Plan** **4**
3.1 What We Are Building (and Not Building) . . . 5
3.2 The Three API Layers . . . 5

**4 Deployment Configurations: Where RDMA Enters the Data Path** **6**
4.1 Configuration A: One User, One Model, Multiple GPUs on One Machine . . . 6
4.2 Configuration B: Multiple Users, One Model, One Machine . . . 7
4.3 Configuration C: Disaggregated Prefill/Decode Across Machines . . . 7
4.4 Configuration D: Multiple Users, One Model, Multiple Machines . . . 8
4.5 Configuration E: Multiple Models, Multiple Machines . . . 8
4.6 Summary: QP and Verbs Mapping by Configuration . . . 9

**5 UCX Issue Landscape and Upstream Contributions** **9**
5.1 Problems Identified . . . 10
5.2 libscuffedrdma's Response . . . 10
5.3 Upstream UCX Contributions . . . 10

**6 KV Cache Compression Pipeline and TurboQuant** **11**
6.1 Three Compression Stages on Orthogonal Axes . . . 11
6.2 TurboQuant . . . 11
6.3 How Compression and Transport Interact . . . 12

**7 Production Systems Context** **12**

**8 Benchmark Results** **13**
8.1 Dual QP Pool: Head-of-Line Blocking . . . 13
8.2 Latency Across Message Sizes . . . 14
8.3 Infrastructure Validation . . . 14

**9 Implementation: Python MVP on SoftRoCE** **14**
9.1 RDMA Connection Lifecycle . . . 14
9.2 Dual QP Pool . . . 15
9.3 WFA Classifier . . . 16
9.4 PMP Bang-Bang Controller . . . 17
9.5 How RDMA Testing Works on SoftRoCE . . . 17

**10 Next Steps** **18**

# 1 Problem Context

Updates 1–3 cover the hardware testbed, problem statement, tensor cache architecture, and transport abstractions. This update adds the implementation, PMP control theory, and benchmark results.

## 1.1 The KV Cache Problem

Without a KV cache, every decode step recomputes attention over the entire sequence (quadratic cost). The KV cache makes decode linear by caching previous tokens' key/value projections.

The cost is memory. For $L$ layers, $K$ KV heads, head dimension $H$, sequence length $S$, FP16:

$$\text{KV cache per request} = 2 \times L \times K \times H \times S \times 2 \text{ bytes}$$

For a 70B model ($L$=80, $K$=8 GQA, $H$=128, $S$=4096): 1.3 GB per request. With 16 concurrent users, 21 GB of KV cache alone on a 24 GB RTX 3090.

In disaggregated serving (prefill on Cerberus, decode on Chimera), this entire cache must cross the 10GbE link (∼1.2 GB/s). On InfiniBand this takes milliseconds. On our testbed it takes seconds. Compression (LoRC, KVTC, TurboQuant) reduces the payload; the dual QP pool reduces the queuing delay. Each targets a different factor in total transfer time (Equation 2).

## 1.2 What This Update Adds

UCX, NIXL, and NCCL treat all traffic equally or route by message size alone. They cannot distinguish a latency-critical decode activation from a background KV prefetch of the same size.

This update adds **tensor-aware RDMA traffic management**. The WFA classifier labels each transfer by its role in the inference pipeline, and the PMP bang-bang controller allocates bandwidth between priority classes. Traffic routes to separate RDMA queue pairs: a hot pool for latency-sensitive transfers, a cold pool for throughput. We validate with real RDMA writes on SoftRoCE.

# 2 Theoretical Foundation

## 2.1 The Scaling Book's Decode Latency Model

Austin et al. [1] derive the decode step time as a function of KV cache size:

$$T_{\text{step}} = \underbrace{\frac{B \times \text{KV}_{\text{size}}}{W_{\text{HBM}}}}_{\text{attention (always BW-bound)}} + \underbrace{\max\left(\frac{2B \times P}{C_{\text{FLOPs}}}, \frac{P_{\text{bytes}}}{W_{\text{HBM}}}\right)}_{\text{MLP (can be compute-bound)}} \tag{1}$$

Decode attention has arithmetic intensity ≈1 (one dot product per KV byte loaded), so it is always memory-bandwidth-bound. The MLP term transitions at $B_{\text{crit}} = C_{\text{FLOPs}}/W_{\text{HBM}}$ [1]. For disaggregated serving, TTFT adds the network transfer:

$$T_{\text{TTFT}} = T_{\text{prefill}} + T_{\text{KV transfer}} + T_{\text{first decode step}} \tag{2}$$

The thesis targets $T_{\text{KV transfer}}$. On a bandwidth-bound 10GbE link, $T_{\text{KV transfer}} = \text{KV}_{\text{bytes}}/W_{\text{net}}$. Every optimization maps to a specific term:

| Tool | Term reduced | Mechanism |
|---|---|---|
| Triton / Pallas / K-Search | $T_{\text{prefill}}$, $T_{\text{decode step}}$ | Better kernel → closer to compute roofline |
| GQA / MQA [26, 27] | $\text{KV}_{\text{size}}$ in Eq. 1, 2 | Fewer KV heads → smaller cache per layer |
| LoRC [8] | $\text{KV}_{\text{size}}$ | SVD on $W_K, W_V$ → lower-dim KV vectors |
| KVTC [9] / TurboQuant [10] | $\text{KV}_{\text{bytes}}$ | Quantize + entropy-code the KV tensors |
| Dual QP pool | $T_{\text{KV transfer}}$ | Prevent head-of-line blocking on 10GbE |
| PMP controller | $T_{\text{KV transfer}}$ | Optimal bandwidth allocation across QPs |
| PTX warp specialization [14] | $T_{\text{prefill}}$ overlap | $\max(T_{\text{math}}, T_{\text{comms}})$ not sum |
| Paged Attention [29] | $\text{KV}_{\text{bytes}}$ | Transfer only populated pages, not padded |
| Layer-by-layer xfer [15] | $T_{\text{KV transfer}}$ overlap | Pipeline prefill layers with RDMA writes |

Table 1: Each optimization maps to a specific term in the TTFT equation.

### 2.2 PMP: Bang-Bang QP Allocation

Zhang et al. [2] reinterpret LoRA matrices as control variables on a dynamical system. PMP [3, 4] gives conditions for finding the optimal control trajectory.

**Definition 1** (RDMA Allocation as Controlled System)**.** *Let* $\mathbf{s}(t) = (q_H(t), q_C(t))$ *be hot/cold queue depths. Control* $u(t) \in [0, 1]$ *sets the hot-path bandwidth fraction:*

$$\dot{q}_H = \lambda_H(t) - u(t) \cdot C \cdot \mu_H, \qquad \dot{q}_C = \lambda_C(t) - (1 - u(t)) \cdot C \cdot \mu_C \tag{3}$$

*where* $\lambda_H, \lambda_C$ *are WFA-classified arrival rates and* $C$ *is 10GbE link capacity.*

**Proposition 1** (Bang-Bang QP Allocation)**.** *The Hamiltonian* $\mathcal{H} = p_H \dot{q}_H + p_C \dot{q}_C$ *is linear in* $u$*, so optimal allocation is bang-bang:* $u^*(t) = 0$ *(route to cold) if* $S > 0$*,* $u^*(t) = 1$ *(route to hot) if* $S < 0$*, where*

$$S = p_H \cdot C \cdot \mu_H - p_C \cdot C \cdot \mu_C$$

*is the switching function and* $p_H, p_C$ *are the costate (adjoint) variables satisfying* $\dot{p}_i = -\partial\mathcal{H}/\partial q_i$*. In the implementation, we approximate the costates by weighted queue depths:* $p_H \approx \alpha \cdot q_H$*,* $p_C \approx \beta \cdot q_C$*, which turns the switching function into something computable from observable state:* $S = \alpha \cdot q_H \cdot C \cdot \mu_H - \beta \cdot q_C \cdot C \cdot \mu_C$*.*

### 2.3 WFA vs. PMP: Complementary Roles

| | WFA Classifier | PMP Controller |
|---|---|---|
| Question | Which class is this tensor? | How much bandwidth per class? |
| Output | hot / warm / cold label | $u^*(t) \in \{0, 1\}$ |
| Timescale | Per-tensor ($\mu$s) | Per-scheduling-epoch (ms) |

## 3 Implementation Plan

libscuffedrdma is the thesis's RDMA middleware. The current prototype is Python on top of pyverbs (rdma-core bindings); a C++ port is a future production goal. The design follows Austin et al.'s roofline framework and decode step time model [1].

## 3.1 What We Are Building (and Not Building)

We are not recreating NIXL or UCX. libscuffedrdma sits between vLLM's `KvConnector` interface and the raw `libibverbs` API, adding tensor-aware classification and priority QP routing that neither NIXL nor UCX provides. The full stack:

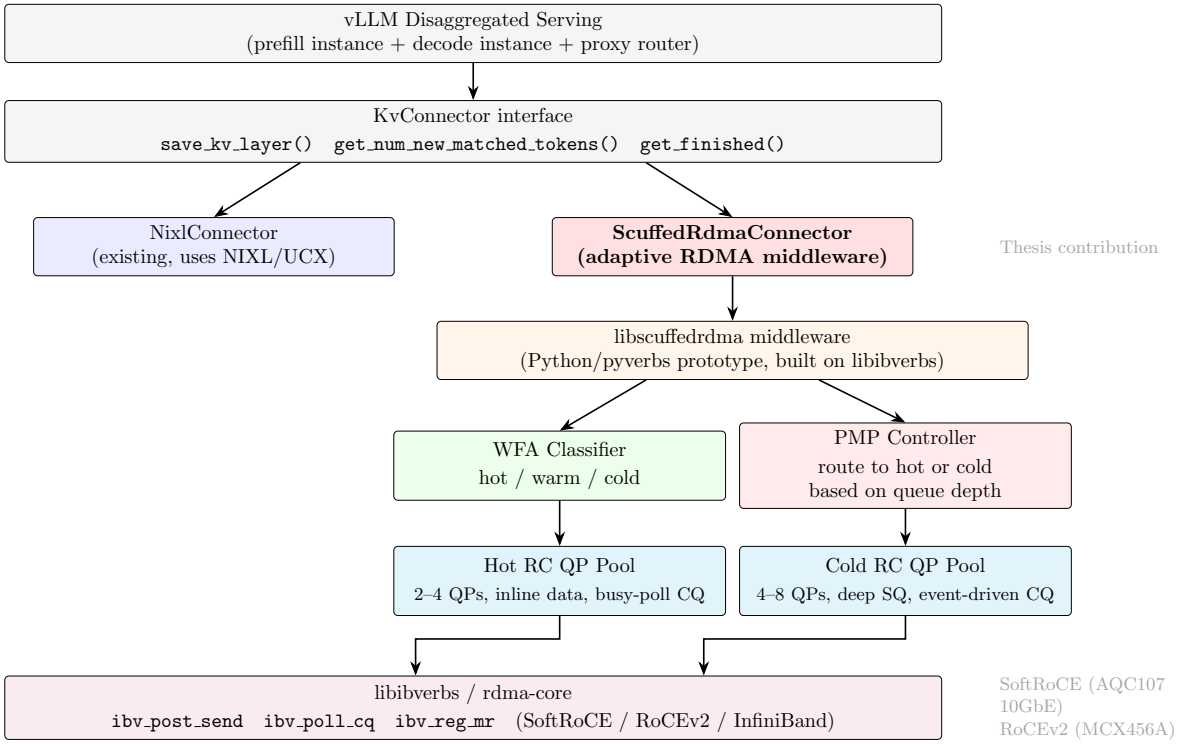


Figure 1: Full system architecture. The middleware (red) adds tensor-aware classification and priority QP routing on top of UCX/libibverbs, replacing NixlConnector's transport layer. Everything above `KvConnector` stays stock vLLM.

## 3.2 The Three API Layers

libscuffedrdma bypasses UCX's transport selection and calls `ibv_post_send` directly. UCX routes messages by size alone (small = eager inline send, large = rendezvous RDMA read). This means UCX cannot distinguish between a latency-critical 64KB decode activation and a 64KB background prefetch. libscuffedrdma's WFA classifier makes that distinction, then posts to the correct QP.

The existing vLLM call chain goes: `save_kv_layer()` → NIXL → UCX → `ibv_post_send()`. We replace the middle: `save_kv_layer()` → WFA classifies → `ibv_post_send()` on the selected QP.

Both QP pools use RC (Reliable Connected) transport. We do not use UD (Unreliable Datagram) because UD has a 4KB MTU limit requiring application-level fragmentation, no delivery guarantees, and poor SoftRoCE performance. RC handles reliability in hardware. The cost is one QP per peer per priority class, which is trivial for a two-node testbed.

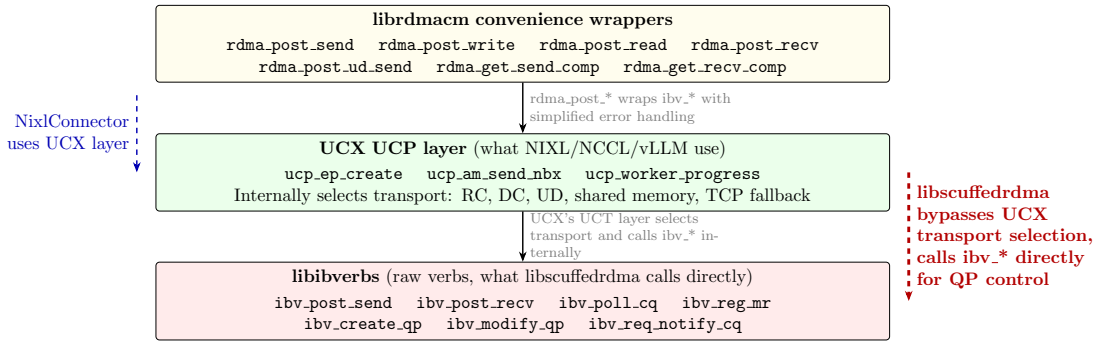


Figure 2: Three RDMA API layers. NixlConnector calls UCX, which internally selects transports. libscuffedrdma bypasses UCX's transport selection and calls `ibv_*` directly, which is necessary for controlling which QP each transfer uses.

| | **Hot Pool (latency)** | **Cold Pool (throughput)** |
|---|---|---|
| SQ depth | 16 entries | 128 entries |
| CQ depth | 16 entries | 256 entries |
| Inline data | Enabled ($\leq$256B in WR) | Disabled (DMA from MR) |
| CQ polling | Busy-poll, dedicated core | Sleep-poll (event-driven in production) |
| Signaling | Every WR | Every $N$th WR |
| Traffic | KV metadata, heartbeats, all-reduce | Bulk KV blocks, weights, gradients |

Table 2: Hot vs. cold RC QP pool configuration. Both pools use RC transport; they differ in tuning.

# 4 Deployment Configurations: Where RDMA Enters the Data Path

Each configuration below traces four data structures through a distributed LLM inference deployment: **model weights** (static, loaded once), **activations** (ephemeral, per-forward-pass), **KV cache** (grows per-token, persists per-request), and **control signals** (metadata, scheduling, consensus). The parallelism strategies below follow directly from Austin et al. [1] Sections 5 and 7.

## 4.1 Configuration A: One User, One Model, Multiple GPUs on One Machine

This is **tensor parallelism** (TP). Each layer's weight matrices are sharded across GPUs. Every GPU holds a slice of every layer. A single user's request flows through all GPUs at every layer.

Each GPU computes its shard of the attention and MLP, then AllReduces the partial result with every other GPU. This happens at every layer. The AllReduce messages are small (a few KB to hundreds of KB) but every GPU blocks waiting for them. This is textbook **hot QP** traffic: small, frequent, latency-critical. On consumer hardware without NVLink, AllReduce goes over PCIe or 10GbE and becomes the bottleneck. When GPU IPC is unavailable, vLLM falls back to NCCL for inter-node AllReduce, which is where libscuffedrdma enters.

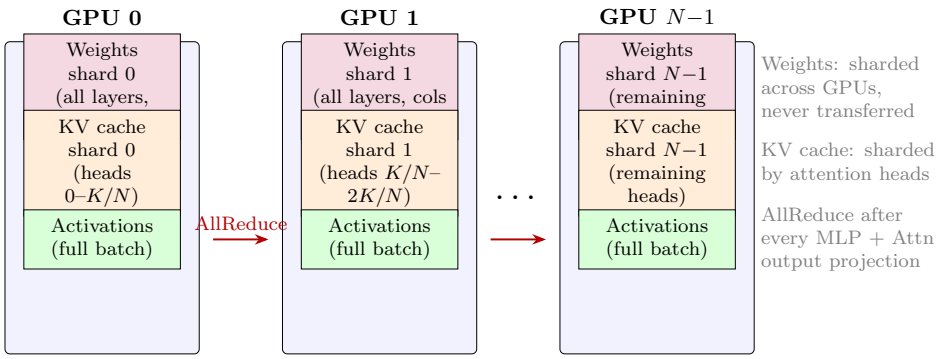


Figure 3: **Config A:** Tensor parallelism, single machine. AllReduce happens over NVLink (intra-node) or PCIe. On consumer hardware without NVLink, PCIe AllReduce is the bottleneck.

### 4.2 Configuration B: Multiple Users, One Model, One Machine

Same as Config A but with continuous batching ($B > 1$ users). Each user has their own KV cache, so total KV memory per GPU scales as $B \times L \times 2 \times K \times H/N$. The RDMA traffic is the same (AllReduce on hot QP) but payloads grow with batch size.

Figure 4: **Config B:** Tensor parallelism with continuous batching. Each GPU holds KV caches for all $B$ concurrent users. AllReduce traffic matches Config A but KV memory scales with batch size.

### 4.3 Configuration C: Disaggregated Prefill/Decode Across Machines

This is the primary target of the thesis. Prefill and decode run on **separate machines**. The prefill node processes the full prompt and generates the KV cache. The prefill node then transfers the KV cache over the network to the decode node, which generates tokens autoregressively.

**Data flow.** The prefill node processes the full prompt in one compute-bound forward pass, producing the KV cache for all layers and the logits for the last token. Both are transferred to the decode node via RDMA. The decode node holds its own copy of model weights, receives the KV cache, and generates tokens autoregressively (memory-bandwidth-bound), appending one token's KV per step.

**What crosses the network.** The KV cache is the dominant transfer. For a model with $L$ layers, $K$ KV heads, head dimension $H$, and sequence length $S$:

$$\text{KV transfer size} = 2 \times \text{bytes} \times H \times K \times L \times S$$

The logits and request metadata are small (a few KB). On 10GbE, the KV transfer is the TTFT bottleneck.

**Dual QP mapping.** **Hot QP:** logits, request metadata, scheduling signals, and consensus heartbeats if a protocol like Raft were added for multi-node coordination. These are small and latency-critical: the decode node cannot begin until it has the logits. **Cold QP:** bulk KV cache blocks.

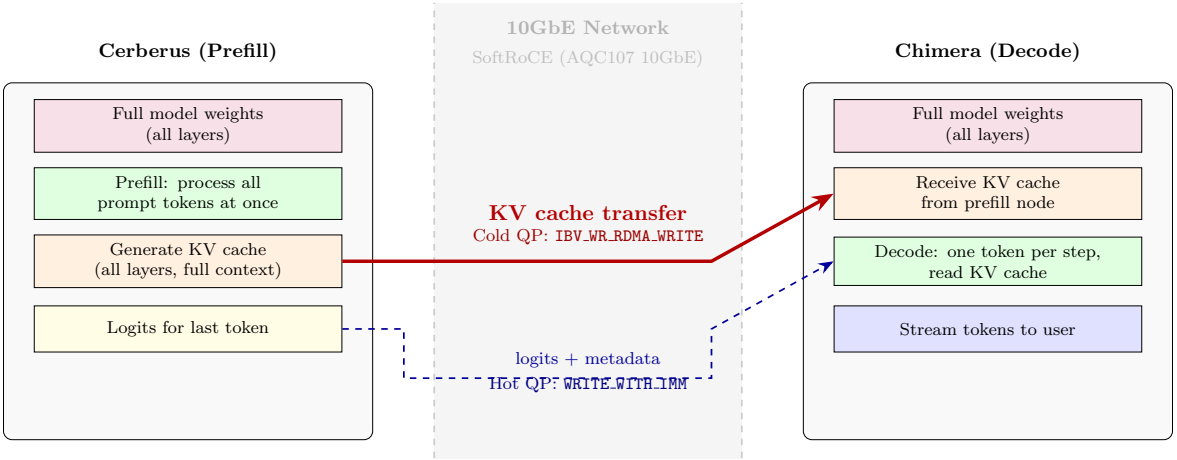


Figure 5: **Config C:** Disaggregated prefill/decode. The thick red arrow is the critical-path RDMA transfer that determines TTFT. The dashed blue arrow carries latency-sensitive metadata on the hot QP.

These are large but can be pipelined layer-by-layer (TransferEngine pattern [15]). The cold QP uses `IBV_WR_RDMA_WRITE` for zero-copy one-sided transfers from prefill GPU memory to decode GPU memory. With GPUDirect RDMA, this is truly zero-copy; on Cerberus/Chimera with SoftRoCE, data stages through host memory.

**Compression reduces the cold QP load.** LoRC [8] reduces $H$ (lower-dimensional KV vectors). KVTC [9] applies entropy coding on the KV tensors. TurboQuant [10] reduces bytes-per-element. Each shrinks the cold QP payload, but the hot QP traffic (metadata, logits) is unaffected since it is already small.

### 4.4 Configuration D: Multiple Users, One Model, Multiple Machines

This combines Configs B and C: disaggregated prefill/decode with continuous batching across multiple users.

Multiple users' KV transfers share the same 10GbE link. Without traffic separation, User A's large KV transfer blocks User B's small metadata. That's the head-of-line blocking problem the dual QP pool solves. The PMP controller allocates bandwidth between pools based on queue depth. With prefix caching (llm-d [17]), some KV blocks are already on the decode node, which changes the traffic pattern the WFA observes.

### 4.5 Configuration E: Multiple Models, Multiple Machines

MoE (Mixture-of-Experts) models and multi-model serving.

In MoE models (e.g. DeepSeek-V3), each token routes to a subset of experts across machines, creating All-to-All traffic at every layer. Popular experts get large predictable batches (cold QP), rare experts get small urgent transfers (hot QP). The WFA classifies by per-expert access frequency. vLLM's elastic expert parallelism (PR #34861 [21]) produces this same hot/cold split when GPUs are dynamically added or removed.

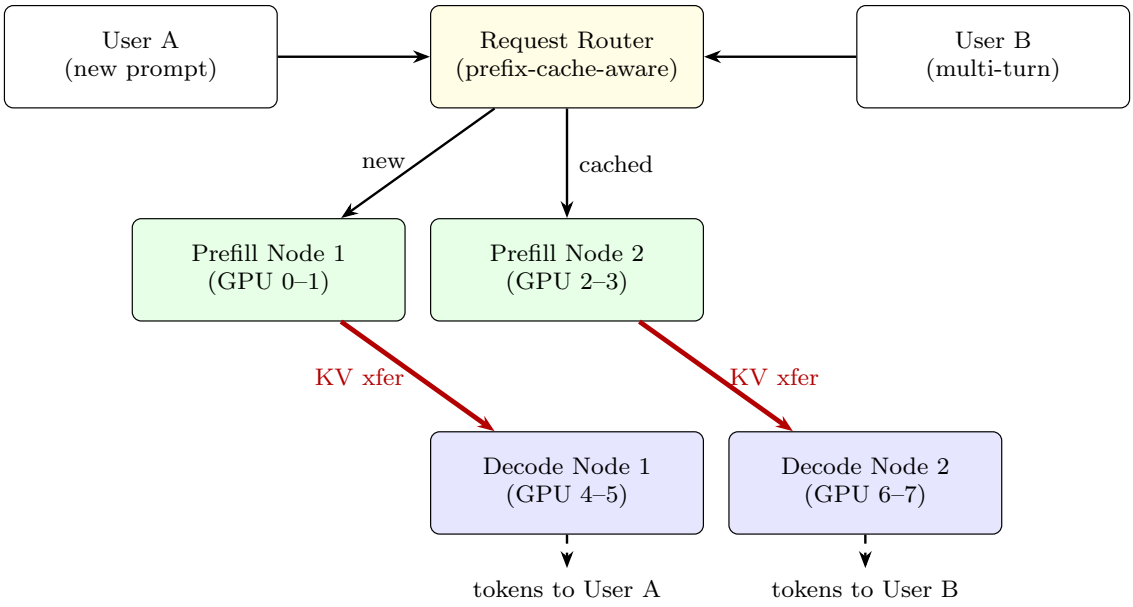


Figure 6: **Config D:** Multi-user disaggregated serving. The router directs requests to prefill nodes based on prefix cache locality (llm-d pattern [17]). KV transfers (red) are independent per-user and can be multiplexed on the same link.

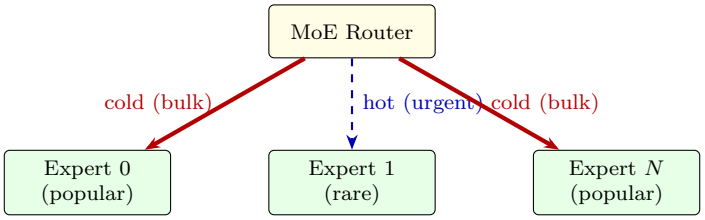


Figure 7: **Config E:** MoE expert parallelism. Popular experts get large batches (cold QP), rare experts get small urgent transfers (hot QP).

### 4.6 Summary: QP and Verbs Mapping by Configuration

**What to optimize first.** Config C (disaggregated prefill/decode, single user) is the simplest configuration that exercises the full dual QP architecture. It has a clear hot/cold separation (metadata vs. KV blocks), a measurable TTFT metric, and maps directly onto the Cerberus/Chimera testbed. Once Config C works, extending to Config D (multiple users) adds the multiplexing problem that makes the PMP controller's bang-bang allocation necessary. Config E (MoE) is the most complex but produces the richest WFA training data due to the bimodal expert access pattern.

## 5 UCX Issue Landscape and Upstream Contributions

UCX is the dominant open-source RDMA transport framework. NCCL, MPI, PyTorch, and Dask all use it. While building libscuffedrdma we ran into recurring architectural issues in UCX's issue tracker, and ended up contributing six fixes upstream. This section covers the problems we hit, the bypass we built in libscuffedrdma, and the patches we sent to UCX.

| Config | Hot QP traffic | Cold QP traffic | Primary verb |
|---|---|---|---|
| A: TP, 1 user, 1 machine | AllReduce fragments | (none) | `WRITE_WITH_IMM` |
| B: TP, $N$ users, 1 machine | AllReduce fragments | (none) | `WRITE_WITH_IMM` |
| C: Disagg P/D, 1 user | Logits, metadata | KV cache blocks | `WRITE` (cold) |
| D: Disagg P/D, $N$ users | Metadata, scheduling | Multiplexed KV | `WRITE` (cold) |
| E: MoE / multi-model | Rare expert tokens | Popular expert tokens | `WRITE_WITH_IMM` |

Table 3: QP and verbs mapping by deployment configuration. Config C is the thesis's primary target; Configs D and E are stretch goals.

### 5.1 Problems Identified

UCX makes three design choices that create friction for LLM inference workloads:

1. **Size-only routing.** Messages below `UCX_RNDV_THRESH` take the eager path; messages above take rendezvous. This creates latency cliffs at the threshold and cannot distinguish a latency-critical decode transfer from a background prefetch of the same size (Issues #10552, #10486, #10532, #4430, #11091).

2. **Shared QP pools.** All traffic shares the same QPs, so a small message can queue behind a large transfer. Processes share UDP source ports (breaking bond hashing, #11004), contend on spinlocks (#11034), and cannot defer QP allocation (#1319).

3. **Opaque transport selection.** Users cannot inspect, override, or query which transport UCX chose (#10652, #6511, #9364, #10608). Multi-rail often degrades performance instead of improving it (#9520, #10669, #10420). The rendezvous path has open bugs since 2017 (#6721, #10568, #1298–#1300).

Additional friction for ML workloads includes silent memory registration failures (#7989), 25–75% TCP slowdown when CUDA is compiled in (#7880), and asymmetric traffic class application (#10325).

### 5.2 libscuffedrdma's Response

libscuffedrdma bypasses UCX's data path entirely. Instead of size-based routing through shared QPs, the WFA classifier assigns each transfer a priority class based on tensor metadata (access frequency, inference phase, layer index), and the PMP controller routes it to a dedicated hot or cold QP. All data-path calls go through `ibv_post_send` directly with pre-registered memory regions. The mapping from tensor to QP is deterministic: there's no opaque scoring or rendezvous handshake, and the hot and cold pools don't share QPs.

Table 4 maps each libscuffedrdma component to the UCX issues it addresses.

### 5.3 Upstream UCX Contributions

Beyond the bypass, we submitted six pull requests to `openucx/ucx` that fix the issues directly (Table 5).

| Component | UCX Issues | What It Does |
|---|---|---|
| WFA classifier | #10552, #10486, #10532, #4430, #11091 | Replaces size-only threshold with tensor metadata classification |
| Hot/cold QP pools | #11004, #11034, #1319 | Dedicated QPs per traffic class eliminate head-of-line blocking |
| Direct `ibv_post_send` | #6721, #10568, #3542, #1298–#1300 | Bypasses rendezvous handshake, rail selection, and wakeup overhead |
| Pre-registered MRs | #10719, #7989, #6264 | One-time `ibv_reg_mr` at startup with explicit error handling |
| PMP controller | #9520, #10669 | Control-theoretic QP allocation replaces static lane scoring |
| Per-QP traffic class | #11252, #10325 | Symmetric TC via direct `ibv_modify_qp` on both endpoints |

Table 4: libscuffedrdma components mapped to the UCX issues they address.

# 6 KV Cache Compression Pipeline and TurboQuant

## 6.1 Three Compression Stages on Orthogonal Axes

**LoRC** [8] applies SVD to $W_K, W_V$, producing lower-dimensional KV vectors at every forward pass without retraining. The progressive strategy retains more singular values in shallow layers (sensitive to truncation) and compresses deep layers aggressively. Zhang et al. report significant compression with negligible quality degradation across LLaMA-8B through 70B [8]. Whether this improves TTFT on our testbed is untested.

LoRC, KVTC [9], and TurboQuant [10] target orthogonal representation axes and compose as a pipeline. The pipeline for a KV vector $k \in \mathbb{R}^d$ is:

$$k \xrightarrow{\text{LoRC (dim)}} k' \in \mathbb{R}^{d'} \xrightarrow{\text{KVTC (redundancy)}} \tilde{k} \in \mathbb{R}^{d'} \xrightarrow{\text{TurboQuant (precision)}} \hat{k} \in \{0, \ldots, 2^b - 1\}^{d'}$$

LoRC reduces $d \to d'$ via SVD on the weight matrices. KVTC applies PCA decorrelation and entropy coding on the generated vectors. TurboQuant reduces the bits-per-element from 16 to as low as 3 while preserving inner product accuracy. Because each stage operates on a different axis of the representation, their effects compound.

## 6.2 TurboQuant

TurboQuant [10] (Zandieh, Mirrokni et al., Google Research) is a two-stage quantization algorithm that does not need per-block calibration data:

1. **PolarQuant**: Apply a random rotation to the KV vector, then quantize. The rotation makes each coordinate's distribution predictable, so the quantizer can be pre-computed once and reused for all vectors. No per-block normalization constants needed.

2. **QJL correction**: The quantization error creates bias in inner products. TurboQuant corrects this by storing a 1-bit sketch of the residual using the Quantized Johnson-Lindenstrauss transform. The corrected inner products are provably unbiased.

| PR | Fixes | Change |
|---|---|---|
| #11304 | #4430 | Use calculated RNDV threshold for `ucp_tag_send_nbr()` instead of hardcoded 256KB |
| #11305 | #1307 | Adaptive TX CQ moderation: force signaled send when CQ credits run low to prevent TX stalls |
| #11306 | #4275 | Allow eager inline sends for host memory when CUDA memory domains are loaded |
| #11307 | #4794 | Add TCP as fallback wireup transport for RC/DC on SoftRoCE where UD is unavailable |
| #11308 | #6254 | Add `ucp_ep_query()` bandwidth estimate for informed application placement decisions |
| #11309 | #10325 | Propagate traffic class in IB address for symmetric QoS on both connection directions |

Table 5: Upstream UCX pull requests submitted to `openucx/ucx` on GitHub.

The key detail for RDMA: TurboQuant preserves *inner product accuracy*, not individual vector accuracy. Attention only needs inner products (query dot key), so this works, but the decode node needs a modified attention kernel that operates on the quantized codes directly. The RDMA layer just moves the smaller compressed bytes.

### 6.3 How Compression and Transport Interact

Compression (TurboQuant, KVTC, LoRC) reduces the bytes that cross the link. The dual QP pool reduces the queuing delay on the link. These target different terms:

$$T_{\text{KV transfer}} = \underbrace{\frac{\text{KV}_{\text{bytes}}}{W_{\text{net}}}}_{\text{compression reduces this}} + \underbrace{T_{\text{queuing delay}}}_{\text{dual QP reduces this}}$$

The RDMA layer is agnostic to whether bytes are compressed. The WFA classifies by size and access pattern, not data content. However, compression shifts the size distribution: blocks that were "large" (cold path) may become "small" (hot path) after compression, so the WFA thresholds should be tuned on compressed traffic.

KVTC (NVIDIA) requires per-model calibration but achieves higher compression. TurboQuant (Google) is model-independent but compresses less. They target different pipeline stages and can be combined. The RDMA transport work is independent of both.

## 7 Production Systems Context

**NVIDIA Dynamo/NIXL** [11]: NIXL selects backends automatically but lacks adaptive QP management. libscuffedrdma fills this gap.

**llm-d v0.5** [17]: UCCL's software transport with fine-grained flow splitting provides greater congestion resilience than UCX [18] by avoiding hardware PFC. Their prefix-cache-aware scheduling reduces TTFT by routing to nodes holding the relevant KV cache.

**TransferEngine** [15]: Bridges common NICs to expose a uniform interface for point-to-point RDMA in LLM systems, implementing one-sided WriteImm operations across multiple NICs per GPU. libscuffedrdma extends this with tensor-aware classification for QP selection.

**Consensus protocols** [25]: If a consensus layer (e.g. Raft) were added, heartbeats would be a textbook hot-path case. On a single 10GbE NIC, a heartbeat blocked behind a large KV transfer could trigger spurious leader elections. No consensus protocol is implemented yet; this is a planned extension.

**Consumer network roofline** [1]: On 10GbE (∼1.2 GB/s), operations that are compute-bound in datacenters (all-reduce, KV transfer) become communication-bound. Software RDMA optimization yields larger relative gains on consumer hardware, where bandwidth is scarce and inefficiencies cannot be masked by raw throughput.

# 8 Benchmark Results

The following results were collected on Chimera (3× RTX 3090, Aquantia AQC107 10GbE, SoftRoCE/rxe0, Ubuntu 24.04, CUDA 12.4, Python 3.12, pyverbs 50.0). All measurements use real RDMA Write operations over the rxe kernel driver.

## 8.1 Dual QP Pool: Head-of-Line Blocking

Table 6: Small-transfer latency under mixed 256B + 1MB workload (1000 iterations, SoftRoCE loopback).

| Scenario | p50 ($\mu$s) | p95 ($\mu$s) | p99 ($\mu$s) |
|---|---|---|---|
| Single QP (baseline) | 13.6 | 15.9 | 17.7 |
| Dual QP (WFA classifier) | 14.2 | 17.7 | 21.4 |
| Dual QP + PMP controller | 14.5 | 18.5 | 23.2 |

The dual QP scenarios are 0.5–5.5$\mu$s slower than the single QP baseline (p50 to p99, PMP enabled). This is expected:

**Why dual QP is slower here.** SoftRoCE is synchronous and single-threaded: transfers execute one at a time, so no head-of-line blocking occurs. The overhead comes from managing two pools instead of one: round-robin QP selection, building separate GlobalRoute and AHAttr structures per QP, running the WFA classifier (access count lookup, phase detection, threshold checks), and running the PMP controller (switching function, history tracking).

**Why this changes with real hardware.** SoftRoCE implements RDMA in the kernel: the CPU handles QP state transitions, packet construction, and completions in software. On hardware (e.g., ConnectX-6), the NIC processes the send queue and generates completions without CPU involvement. This changes two things:

- **Base latency drops.** Hardware RDMA on ConnectX typically achieves 1–2$\mu$s for small messages vs our 13–14$\mu$s on SoftRoCE. The measured dual-QP overhead (0.5–5.5$\mu$s, including classification, routing, and PMP) becomes a larger fraction of the transfer time but is still small relative to the cold path.
- **QP isolation becomes real.** With hardware offload, the NIC processes each QP's send queue independently. A 1MB write occupying the cold QP does not block the hot QP because

the NIC has separate hardware contexts for each. With SoftRoCE, the kernel processes everything sequentially regardless of which QP it belongs to. Hardware RDMA is where the dual QP architecture pays off: concurrent producers can post to hot and cold QPs simultaneously, and the NIC drains them in parallel.

**When head-of-line blocking appears.** The benefit requires at least one of: (a) multiple threads posting transfers concurrently, so one thread's 1MB cold write is in-flight while another thread's 256B hot write needs to go out; or (b) async posting with queue depth $> 0$, where multiple work requests are queued before any complete. In both cases, sharing a single QP means the 256B message waits behind the 1MB message in the send queue. Separate QPs eliminate this wait. The concurrent workload benchmark and cross-node testing on real hardware are the next steps to measure this.

### 8.2 Latency Across Message Sizes

Table 7: libscuffedrdma latency vs. message size (50 iterations per size, SoftRoCE loopback). Sizes marked † correspond to UCX eager/RNDV protocol transition boundaries.

| Size (B) | p50 ($\mu$s) | p95 ($\mu$s) | p99 ($\mu$s) | Pool |
|---:|---:|---:|---:|---|
| 64 | 12.6 | 19.4 | 48.1 | hot |
| 256 | 12.9 | 14.2 | 24.6 | hot |
| 1024 | 12.8 | 13.6 | 24.8 | hot |
| 3000† | 18.7 | 22.4 | 30.8 | hot |
| 8192† | 32.7 | 41.3 | 44.5 | hot |
| 32768† | 98.4 | 110.6 | 112.8 | hot |
| 65536† | 183.8 | 206.4 | 278.2 | hot |
| 262144 | 704.9 | 732.4 | 742.3 | hot |
| 524288 | 1414.5 | 1450.4 | 1459.1 | cold |
| 1048576 | 2822.6 | 2863.6 | 3104.0 | cold |

The latency curve is smooth and monotonic across all message sizes. There is no cliff at the 8KB, 30KB, 32KB, or 64KB boundaries where UCX switches between eager, zcopy, and rendezvous protocols (issues #10552, #10486, #10532). The WFA classifier switches from hot to cold at 512KB. The configured size threshold is 256KB, but frequently-accessed tensors get promoted to hot, pushing the effective crossover to 512KB.

### 8.3 Infrastructure Validation

## 9 Implementation: Python MVP on SoftRoCE

The Python implementation validates the dual QP architecture on real RDMA hardware before porting to C++. All code uses pyverbs (Python bindings for rdma-core) against the SoftRoCE kernel driver (`rdma_rxe`) on Chimera's Aquantia AQC107 10GbE NIC.

### 9.1 RDMA Connection Lifecycle

Every RDMA transfer requires a connected queue pair (QP). A QP starts in RESET state and must transition through three states before it can send data. Both sides must go through these

Table 8: Chimera infrastructure state (March 2026).

| Component | Status |
|---|---|
| RDMA device | rxe0 ACTIVE on enp69s0 (10Gbps, MTU 9000) |
| GPUs | 3× NVIDIA GeForce RTX 3090 |
| CUDA | 12.4.131 |
| UCX | 1.16.0 (verbs, mlx5-dv, RC, DC, UD) |
| nvidia-peermem | Available (v550.144.03), not loaded |
| NIC | Aquantia AQC107 10GbE |
| Python / pyverbs | 3.12.3 / 50.0 |

transitions. Before connecting, they swap QP numbers and GIDs over a separate TCP connection (RDMA has no discovery mechanism).

```
SETUP:
  ctx = open_device("rxe0")
  pd  = alloc_protection_domain(ctx)
  cq  = create_completion_queue(ctx, depth)
  qp  = create_queue_pair(pd, cq, type=RC)

CONNECT (both sides, with remote info exchanged over TCP):
  qp.to_init(port=1, access=LOCAL_WRITE|REMOTE_WRITE|REMOTE_READ)
  qp.to_rtr(remote_qpn, remote_gid, mtu=1024)
  qp.to_rts(timeout=14, retry=7)

REGISTER MEMORY:
  mr = register_region(pd, size, access=LOCAL_WRITE|REMOTE_WRITE)
  -- mr.buf is the virtual address, mr.lkey/rkey are access keys

TRANSFER (one-sided, no remote CPU involvement):
  post_send(qp, opcode=RDMA_WRITE, local_buf, remote_addr, remote_rkey)
  poll_cq(cq) -- blocks until NIC confirms completion
```

The key insight: `RDMA_WRITE` writes directly into remote memory using the remote address and rkey. The remote CPU never sees the transfer. This is what makes RDMA fast, and why QP isolation matters: if hot and cold traffic share a QP, the NIC processes them in FIFO order.

### 9.2 Dual QP Pool

The `DualQPPool` class creates two separate pools of QPs on the same protection domain. Each pool has its own completion queues, so polling one pool never touches the other.

```
class DualQPPool:
  open():
    ctx = open_device("rxe0")
    pd  = alloc_protection_domain(ctx)
    gid = query_gid(port=1, index=1)  -- link-local IPv6
```

```
    for i in 0..n_hot:
      hot_qps[i] = create_qp(pd, cq_depth=16,  max_send=16)
    for i in 0..n_cold:
      cold_qps[i] = create_qp(pd, cq_depth=256, max_send=128)

  post_write_hot(buf, remote_addr, remote_rkey, length):
    qp = hot_qps[round_robin++ % n_hot]
    t0 = now()
    post_rdma_write(qp, buf, remote_addr, remote_rkey, length)
    busy_poll(qp.cq)           -- spin until completion
    return (now() - t0) in microseconds

  post_write_cold(buf, remote_addr, remote_rkey, length):
    qp = cold_qps[round_robin++ % n_cold]
    t0 = now()
    post_rdma_write(qp, buf, remote_addr, remote_rkey, length)
    sleep_poll(qp.cq, 50us)   -- poll, sleep 50us, repeat
    return (now() - t0) in microseconds
```

Hot QPs busy-poll (spin on the CQ until a completion arrives). This burns CPU but gives minimum latency. Cold QPs sleep-poll (check, sleep 50$\mu$s, check again). This frees CPU for other work at the cost of up to 50$\mu$s added latency per poll cycle.

### 9.3 WFA Classifier

The classifier decides which pool handles each transfer. It examines three things: transfer size, access frequency, and inference phase.

```
classify(tensor_id, size_bytes, layer_idx):
  count = increment_access_count(tensor_id)
  phase = detect_phase(layer_idx)

  -- Size routing (primary decision)
  if size < 4KB:       queue = HOT
  elif size > 256KB:   queue = COLD
  else:
    if count >= 10:    queue = HOT    -- frequently accessed
    else:              queue = COLD   -- infrequent

  -- Phase adjustment
  if phase == PREFILL and queue == HOT and size > 2KB:
    queue = COLD       -- prefill is bulk, bias cold
  if phase == DECODE and queue == COLD and count >= 5:
    queue = HOT        -- decode is latency-sensitive, bias hot

  return queue

detect_phase(layer_idx):
```

```
recent = last 8 layer indices seen
if recent == [L, L+1, L+2, ...]:  return PREFILL
if recent == [L, L, L, ...]:      return DECODE
else:                              return UNKNOWN
```

Phase detection works by observing layer access order. Prefill sweeps sequentially through all layers (L0, L1, L2, ...). Decode repeatedly accesses the same layer as it generates tokens. The classifier figures out the phase on its own, so the inference engine doesn't have to signal it.

### 9.4 PMP Bang-Bang Controller

The controller monitors queue depths and overrides the classifier when one pool is backing up relative to the other.

```
decide(hot_depth, cold_depth):
  -- Approximate costates by weighted queue depths
  p_hot  = alpha * hot_depth    -- alpha=2.0
  p_cold = beta  * cold_depth   -- beta=1.0

  -- Switching function
  S = p_hot * C * mu_hot - p_cold * C * mu_cold

  if S > deadband:    return COLD   -- hot congested
  if S < -deadband:   return HOT    -- cold congested
  else:               return last_decision  -- hysteresis
```

The deadband (0.1) prevents chattering: without it, the controller would flip between HOT and COLD every cycle when the queues are nearly balanced. The weights are asymmetric ($\alpha$=2, $\beta$=1) because hot traffic is latency-sensitive, so draining the hot queue matters more.

### 9.5 How RDMA Testing Works on SoftRoCE

SoftRoCE (`rdma_rxe` kernel module) implements RDMA verbs in software over any Ethernet NIC. The kernel handles QP state, packet construction, and completions, so latencies are higher (tens of microseconds vs. single-digit). The API is identical though, so code that works on SoftRoCE runs on ConnectX hardware unchanged.

**Loopback testing.** Two `DualQPPool` instances on the same machine, both using `rxe0`. QP info exchanged in-process, all QP pairs connected, then RDMA writes measured for completion latency.

```
pool_a = DualQPPool("rxe0", n_hot=2, n_cold=2)
pool_b = DualQPPool("rxe0", n_hot=2, n_cold=2)
pool_a.open();  pool_b.open()

pool_a.connect_all(pool_b.get_local_info())
pool_b.connect_all(pool_a.get_local_info())

send_buf = pool_a.register_buffer("send", 1MB)
recv_buf = pool_b.register_buffer("recv", 1MB)
```

```
for i in 0..1000:
  if i is even:
    pool_a.post_write_hot(send_buf, recv_buf.addr,
                          recv_buf.rkey, 256B)
  else:
    pool_a.post_write_cold(send_buf, recv_buf.addr,
                           recv_buf.rkey, 1MB)
```

**Cross-node testing.** For Chimera ↔ Cerberus, each machine runs one pool. QP info exchanges over TCP (192.168.1.x management network), RDMA traffic flows over the 10.0.0.x subnet (direct 10GbE cable). Both machines need SoftRoCE active and an IPv4-mapped GID in the RDMA GID table (populated by adding an IPv4 address to the RDMA-facing interface). A few things bit us along the way:

- pyverbs on Ubuntu 24.04 uses property setters for `QPAttr`, `set_wr_rdma(rkey, addr)` for RDMA work requests, and `cq.poll()` returns `(count, list)`. These differ from the rdma-core docs, so we found them by trial and error.
- SoftRoCE needs a routable GID for cross-node connections. Link-local IPv6 works for loopback but not between machines. Adding an IPv4 address populates an IPv4-mapped GID at a higher index.
- The `rdma_rxe` module enters a zombie state if processes exit without closing resources, requiring a reboot. Production code has to handle cleanup carefully.
- In synchronous single-threaded SoftRoCE, head-of-line blocking doesn't appear because each operation finishes before the next starts. The dual QP benefit only shows up with concurrent producers or hardware CQ offload.

## 10 Next Steps

1. Multi-threaded concurrent workload to demonstrate head-of-line blocking elimination (cross-node RDMA already works; concurrent producers are the remaining piece).
2. vLLM KvConnector integration so prefill runs on Cerberus and decode runs on Chimera over the libscuffedrdma transport, with scuffedQuant compression in the path.
3. Port pyverbs transport to Rust or C++ to remove the GIL bottleneck for concurrent QP polling.

## References


[1] J. Austin et al., "How to scale your model," Google DeepMind, 2025. `https://jax-ml.github.io/scaling-book/`

[2] C. Zhang, J. Cheng, and Q. Li, "An optimal control view of LoRA and binary controller design for vision transformers," *Proc. ICML*, 2024. arXiv:2406.06958.

[3] L. S. Pontryagin, *Mathematical Theory of Optimal Processes*. Routledge, 2018.

[4] R. Bellman, I. Glicksberg, and O. Gross, "On the 'bang-bang' control problem," *Q. Appl. Math.*, vol. 14, no. 1, pp. 11–18, 1956.

[5] D. Liberzon, *Calculus of Variations and Optimal Control Theory*. Princeton Univ. Press, 2011.

[6] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.

[7] S. Cao and I. Stoica, "K-Search," arXiv:2602.19128, 2025.

[8] R. Zhang et al., "LoRC: Low-rank compression for LLMs KV cache with a progressive compression strategy," arXiv:2410.03111, 2024.

[9] M. Staniszewski and T. Łańcucki, "KV cache transform coding for compact storage in LLM inference," arXiv:2511.01815, 2024.

[10] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," Google Research, arXiv:2504.19874, 2025. Uses PolarQuant (arXiv:2502.02617) and QJL.

[11] NVIDIA, "Dynamo: Open-source inference optimization framework," GTC 2025.

[12] P. Tillet, H. T. Kung, and D. Cox, "Triton," *Proc. MAPL*, 2019.

[13] R. Dyro, "Pallas-Triton kernels and kernel auto-tuning," 2025. `https://robertdyro.com/articles/pallas-triton_kernels/`

[14] dhmnr, "Introduction to PTX optimization," 2025. `https://dhmnr.sh/posts/intro-to-ptx-optimization/`

[15] L. Chen et al., "RDMA point-to-point communication for LLM systems," arXiv:2510.27656, 2025.

[16] Mistral AI, "Heaps do lie: Debugging a memory leak in vLLM," 2025.

[17] Red Hat, Google, IBM, "llm-d v0.5: Sustaining performance at scale," 2026.

[18] NVIDIA, "HPC-X v2.15: UCX Framework Library."

[19] AMD, "MaxText-Slurm," ROCm Blog, 2025.

[20] vLLM, "vLLM TPU: Unified backend supporting PyTorch and JAX on TPU," 2025.

[21] vLLM Project, "PR #34861: Elastic expert parallelism," 2026.

[22] L. Chen, "Harnessing 3200 Gbps network (1): RDMA and EFA," 2024.

[23] R. Dyro, "torch2jax," 2025. `https://robertdyro.com/torch2jax/roadmap/`

[24] SemiAnalysis, "Nvidia Blackwell perf TCO analysis," 2025.

[25] R. Sharoon, "Paxos vs Raft vs ZAB," Medium, 2024.

[26] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[27] J. Ainslie et al., "GQA: Training generalized multi-query transformer models from multi-head checkpoints," arXiv:2305.13245, 2023.

[28] D. Benveniste, "How to improve decoding latency with faster self-attention mechanisms," The AiEdge Newsletter, 2025.

[29] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," *Proc. SOSP*, 2023.

[30] Apple, "macOS 26.2: RDMA over Thunderbolt," 2025. HN: `https://news.ycombinator.com/item?id=46248644`

[31] D. Barak, "Tips and tricks to optimize your RDMA code," RDMAmojo, 2013. `https://www.rdmamojo.com/`

[32] B. Rothenberger et al., "ReDMArk: Bypassing RDMA security mechanisms," *Proc. USENIX Security*, 2021.

[33] FlashInfer, "Kernel library for LLM serving," 2025. `https://github.com/flashinfer-ai/flashinfer`